Homework 4 Report

Baran Ecertemel 0087279

Phase 1

How I inspected and analyzed the code: I started by disassembling the ctarget executable using the objdump -d ctarget > ctarget.asm command. I analyzed the control flow of the getbuf function, which is normally called by the test function. The getbuf function reads user input using the Gets() function. Since Gets() simply copies byte sequences and does not perform bounds checking on the input size, it is highly vulnerable to a buffer overflow attack. My objective was to hijack the program's control flow. By intentionally overrunning the allocated buffer, I could overwrite the saved return address on the stack. Consequently, when getbuf executes its retq instruction, it would pop my injected malicious address into the instruction pointer (%rip) instead of returning to the test function, redirecting execution to touch1.

How I determined the number of padding bytes to use: By examining the disassembled machine code of the getbuffunction, I observed the instruction sub $0x20, %rsp. This instruction allocates exactly 32 bytes for the buffer on the stack. Furthermore, at the very beginning of the function, the instruction push %rbp is executed, which pushes the 8-byte base pointer onto the stack. The return address that I needed to overwrite is located immediately after this saved base pointer. Therefore, the total offset required to reach the return address is the sum of the buffer size and the saved base pointer size: 32 bytes + 8 bytes = 40 bytes.

How I found the addresses: To redirect the program to the touch1 function, I needed to find its exact starting address in memory. I used the command line tool objdump combined with grep (objdump -d ctarget | grep touch1). The output revealed that the touch1 procedure starts at the memory address 0x4018cc.

How I crafted my payload: To construct the exploit string (saved as ctarget1.txt), I first generated 40 bytes of padding. I used the arbitrary hex value be repeated 40 times to completely fill the allocated buffer space and overwrite the saved %rbp. Immediately following this padding, I appended the target memory address of touch1 (0x4018cc). Because

the x86-64 architecture utilizes Little-Endian byte ordering, I had to write the address in reverse byte order. Thus, the address was appended as cc 18 40 00 00 00 00 00. Finally, I passed this text file through the provided hex2raw utility to convert the hex representations into a raw byte string, and piped the output into the vulnerable program using ./hex2raw < ctarget1.txt | ./ctarget -q.

Phase 2

How I inspected and analyzed the code: In this phase, the objective was not only to redirect the program's execution to the touch2 function but also to pass a specific argument (my personal cookie value: 0x36d52185) to it. According to the x86-64 calling conventions, the first integer or pointer argument to a function is passed via the %rdi register. Because there was no existing code in the program that would set %rdi to my cookie and then call touch2, I had to inject my own custom assembly instructions into the buffer. The control flow hijack involved overwriting the return address of getbuf so that it jumps to the beginning of the buffer to execute my injected shellcode. This shellcode would then initialize the %rdi register and transfer control to touch2.

How I determined the number of padding bytes to use: The total offset required to overwrite the return address remained the same as in Phase 1: 40 bytes (32 bytes allocated for the buffer via sub $0x20, %rsp plus 8 bytes for the saved %rbp). However, in this phase, the beginning of the buffer was occupied by the injected shellcode. My compiled shellcode consisted of exactly 13 bytes of machine code. Therefore, the required padding was the difference between the total offset and the shellcode length: 40 bytes - 13 bytes = 27 bytes.

How I found the addresses: This phase required two crucial addresses:

-The target function (touch2): Using objdump -d ctarget | grep touch2, I found that the touch2 function starts at the address 0x4018f8.

-The starting address of the buffer: To make the program jump to my shellcode, I needed the exact runtime address of the buffer. I used the GNU Debugger (GDB) for dynamic analysis. Knowing that the Gets(buf) function call passes the buffer's address as its first argument in the %rdi register, I set a breakpoint right before the callq <Gets> instruction at 0x4018c0. Upon running the program and hitting the breakpoint, I inspected the register using the print /x $rdi command. GDB revealed the buffer's starting address to be 0x5566f768.

How I crafted my payload: To create the payload (ctarget2.txt), I first wrote the required shellcode in an assembly file (code.s):

mov $0x36d52185, %rdi

pushq $0x4018f8

ret

I compiled this code using gcc and extracted the raw machine bytes using objdump, which yielded a 13-byte hex sequence: 48 c7 c7 85 21 d5 36 68 f8 18 40 00 c3. The final exploit string was crafted by sequentially placing three components:

-The 13-byte shellcode at the very beginning of the buffer.

-Exactly 27 bytes of padding to completely fill the 40-byte space and reach the return address.

-The starting address of the buffer (0x5566f768) appended in Little-Endian format.
When getbuf completes, it pops this buffer address into the instruction pointer, safely landing on my shellcode, passing the cookie into %rdi, and flawlessly invoking touch2.

Phase 3

How I inspected and analyzed the code: In Phase 3, the goal was to redirect the execution to the touch3 function. However, unlike Phase 2, touch3 expects a string representation of my cookie as its argument, which means %rdi must hold a pointer to the memory address where this string is stored. A critical observation in the assignment instructions was that touch3 calls hexmatch and strncmp, which allocate local variables on the stack. If I placed my cookie string inside the vulnerable 40-byte buffer, the execution of these subsequent functions would overwrite and corrupt my string data. Therefore, the control flow hijack required a strategic memory layout: placing the injected shellcode inside the buffer, but storing the string data in a safe zone higher up on the stack, beyond the overwritten return address, where it would remain untouched by hexmatch.

How I determined the number of padding bytes to use: The baseline offset to reach the return address remained 40 bytes (32 bytes for the allocated buffer plus 8 bytes for the saved %rbp). My injected shellcode was exactly 13 bytes long. To fill the remainder of the

buffer up to the return address, I utilized 27 bytes of padding (40 - 13 = 27 bytes). Additionally, I had to determine the placement of the string. Since the string was placed immediately after the 8-byte return address, its exact offset from the beginning of the buffer was calculated as 48 bytes .

How I found the addresses: This phase required tracking three distinct memory addresses:

-The target function (touch3): Using objdump -d ctarget | grep touch3, I identified the starting address of touch3 as 0x401a13.

-The buffer address: I reused the buffer's starting address obtained via GDB in Phase 2, which was 0x5566f768.

-The string pointer address: To calculate the absolute memory address of the "safe zone" where my string would reside, I added the 48-byte offset (0x30) to the base buffer address: 0x5566f768 + 0x30 = 0x5566f798.

How I crafted my payload: First, I converted my cookie into its ASCII hex representation, appending a null terminator as required by C strings. This resulted in the byte sequence: 33 36 64 35 32 31 38 35 00. Next, I crafted the custom shellcode in an assembly file to load the calculated string address into %rdi and jump to touch3:

mov $0x5566f798, %rdi

pushq $0x401a13

ret

Compiling this generated the 13-byte machine code: 48 c7 c7 98 f7 66 55 68 13 1a 40 00 c3. The final payload was meticulously assembled in the following order:

-The 13-byte shellcode.

-27 bytes of padding (BE) to reach the return address.

-The buffer's starting address (0x5566f768), written in Little-Endian format to overwrite the instruction pointer.

-The hex sequence of the cookie string, placed exactly at the safe address 0x5566f798. This robust structure successfully passed the string pointer to touch3 without risking data corruption during stack operations.

Phase 4

How I inspected and analyzed the code: Unlike the ctarget executable, rtarget incorporates modern security defenses: it randomizes the stack positions (ASLR) and marks the stack memory region as non-executable. Consequently, injecting and executing custom shellcode on the stack was no longer possible. To circumvent these protections, I employed Return-Oriented Programming. The goal remained the same as Phase 2: to execute the touch2 function and pass my personal cookie as an argument via the %rdi register. Since I could not write new code, my strategy was to hijack the control flow by chaining together existing short instruction sequences already present in the executable. Each gadget ends with a retq instruction, allowing the program to continuously pop the next address from the stack and jump to it. I determined that I needed a two-gadget chain: the first to pop my cookie from the stack into a general-purpose register like %rax, and the second to move that value from %rax into %rdi before returning into touch2.

How I determined the number of padding bytes to use: The initial buffer overflow vulnerability within the getbuffunction remained identical to the previous phases. The buffer allocation and the saved base pointer still occupied a combined total of 40 bytes. Therefore, exactly 40 bytes of padding were required to overrun the buffer and reach the return address. Unlike the code injection attacks, no shellcode was placed inside this buffer space; the entire payload following the padding consisted solely of 64-bit memory addresses and data values.

How I found the addresses: This phase required meticulously hunting for byte encodings within the designated gadget farm. I used objdump -d rtarget to analyze the executable.

-Gadget 1 (popq %rax; retq): I needed the byte sequence 58 c3. By inspecting the disassembled code, I discovered this sequence unintentionally hidden inside another instruction within the <addval_156> function. The instruction lea -0x3c6f6fa8(%rdi),%eax translated to the bytes 8d 87 58 90 90 c3. By intentionally jumping two bytes into this sequence, the CPU interprets it as 58 90 90 c3 (popq %rax; nop; nop; retq). This gadget started at address 0x401ac0.

-Gadget 2 (movq %rax, %rdi; retq): I searched for the byte sequence 48 89 c7 c3. I found this hidden within the <addval_457> function. The instruction lea -0x3876b752(%rdi),%eax encoded to 8d 87 ae 48 89 c7 c3. By jumping three bytes into this instruction, I isolated the exact sequence I needed starting at address 0x401acf.

-Target Function (touch2): Using objdump -d rtarget | grep touch2, I confirmed that touch2 was located at address 0x4018f8.

How I crafted my payload: I constructed the final payload (rtarget2.txt) as a continuous ROP chain. First, I filled the buffer with 40 bytes of padding (BE). Next, I overwrote the initial return address with the address of Gadget 1. Immediately following this address, I placed my cookie data. When getbuf returns, it jumps to Gadget 1, and its popq instruction pulls this cookie data off the stack directly into %rax. Following the data, I appended the address of Gadget 2. Gadget 1's retq pops this next address, transferring execution to Gadget 2, which copies the cookie from %rax to %rdi. Finally, Gadget 2's retq pops the last address in my payload: the starting address of touch2. This orchestrated sequence successfully passed the argument and executed the target function without relying on an executable stack.